

UE5 Master Material

Task: Build a single master material for a “worn painted metal” surface, driven entirely by **Material Instances** (no editing the master material per-asset).

Requirements:

- Base Color, Normal, and a **packed ORM texture** (can either make these yourself in substance designer or source outside textures)
- At least 4 exposed parameters that meaningfully change the look of the material
- Must be built in **UE 5.4** or above

UE5 Documentation

Documentation:

This part of the documentation serves to identify all the resources and notes I took on learning the process of creating a Material Instance in Unreal Engine 5.4

Packed ORM texture = https://youtu.be/sh6le_tH8gA?si=RmKKtOncLXGOPFhB

An ORM texture is a single texture that stores 3 different material maps, with the standard RGB, which corresponds to Ambient Occlusion(AO), Roughness, and Metallic respectively.

Red → Ambient Occlusion

Green → Roughness

Blue → Metallic

Material Instances = [Creating and Using Material Instances in Unreal Engine | Unreal Engine 5.8 Documentation | Epic Developer Community](#)

A material Instance follows the parent, child hierarchy. It allows for multiple instances(or children) to use the same material.

- Uses a concept called **inheritance**, where “the properties of the parent material are passed onto its children”.

Parameters of the material instance are available to the artist to change the material.

The 2 Steps

- Create a parameterized material in Unreal Engine
- Create a material instance constant in the **content browser**

To make material attributes editable, you must use **parameter** nodes instead of regular **Material Expression** nodes.

You can do this by:

→ Search for the word ‘parameter’ in the **Material Palette**

Parameter Shortcuts

Two of the most commonly used Material parameters have keyboard shortcuts in the Material Editor:

1. **Scalar Parameter** - Hold the **s** key and **left-click** in the Material graph to place a Scalar Parameter. A Scalar Parameter is the parameterized version of a Constant, and contains a single numeric value.
2. **Vector Parameter** - Hold the **v** key and **left-click** in the Material graph to place a Vector Parameter. A Vector Parameter is the parameterized version of a Constant 4 Vector and contains four floating-point values.

Use the shortcut or palette to place a Scalar parameter in your Material graph:

S + Left-mouse click = places a Scalar Parameter(contains a single numeric value)

V + Left-moust click = places a Vector parameter(Contains four floating point values)

You can limit the range or set a detail value of the Material instance in the Details panel.

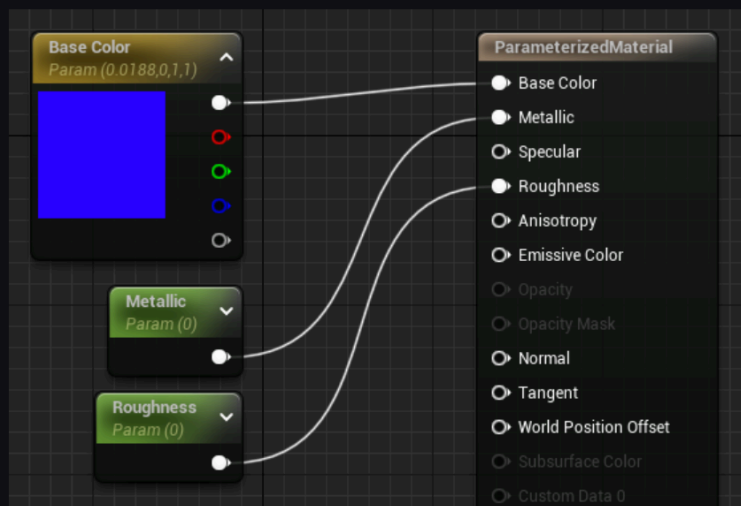
The **roughness** and **metallic** values can only accept a value of 0 - 1, so a good practice is to limit the range from 0 to 1.

Ctrl + D = duplicate a node

You can also convert a non-parameterized value, into a parameterized one by right clicking the node and selecting '**Convert to Parameter**'.

Parameterized Material Example

Your Material graph should now contain a Vector parameter plugged into BaseColor and two Scalar parameters, plugged into the Metallic and Roughness inputs:



These three parameters will allow you to change the color, roughness, and metallic properties of the Material once you create a Material instance.

Click **Apply** in the upper left corner of the [Material Editor](#) to compile the Material, and then click **Save**.

Once you have a **parameterized material**, you can then convert it into **Material Instance**.

- Right click the Material you created in the **Content Browser**, then select '**Create Material Instance**'

You can rename the material using the keybind **F2**

Double click the material in the **Content Browser** to open the material instance editor.

- You can customize **material parameters** in the material instance editor
- You still need to go back to the original Material Graph to change the Parameters of the materials

You can create **parameter** groups by editing the Material, not the Material instance.

- Type a name into the Group panel under **Material Expression**

ORM Texture:

ORM textures use 3 values(RGB) to condense the data needed to support the textures in game engines. They utilize Ambient Occlusion, Roughness, and Metallic as the material maps.

ORM Texture:  87: Let's understand ORM or ARM textures in a PBR material.

Day-By-Day Documentation:

This is my Day-By-Day documentation which highlights my achievements and planning process going forward through the learning process.

9/17/26

At the moment, there are 3 things I need to research more extensively. I don't have much experience with Unreal Engine so I need to get up to speed with what knowledge I do have.

I also haven't had prior experience with textures, so learning how to make them and use them in Unreal Engine will be a fun challenge.

Currently Need to Learn:

- Substance Painter(Unless I make the texture in Blender)
- How to properly implement Material Instancing

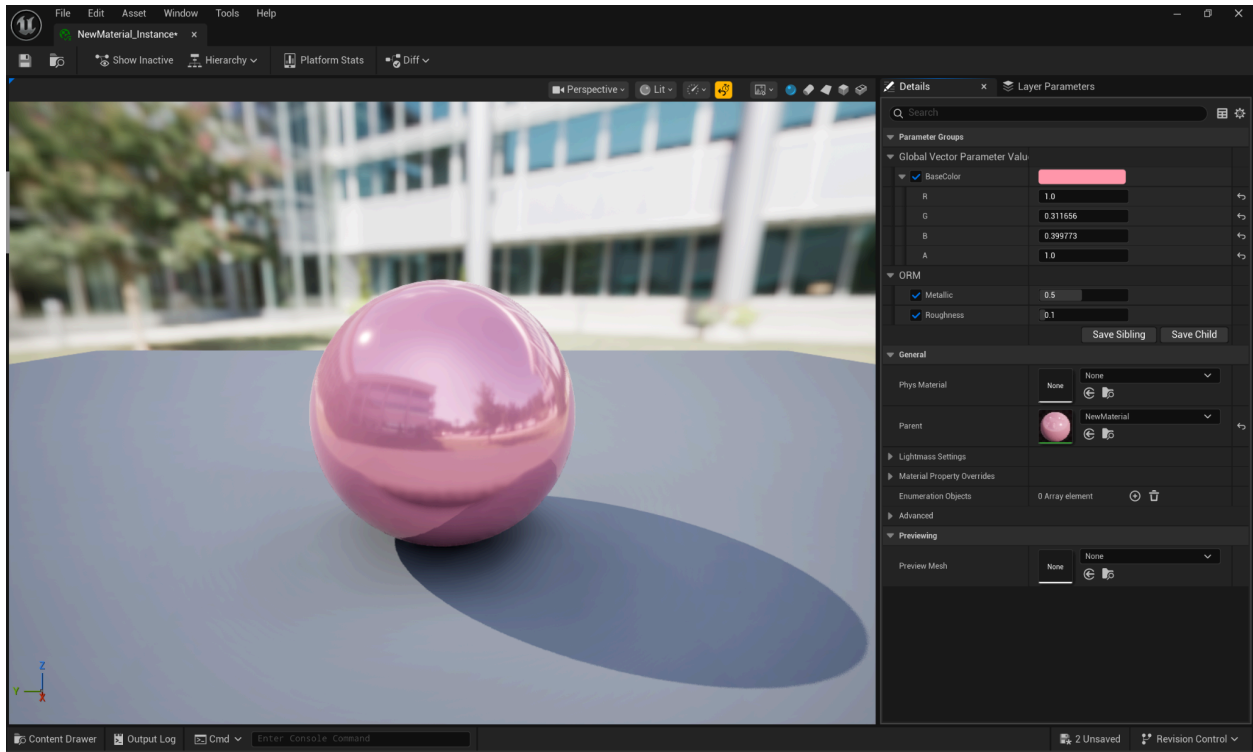
My goal right now with the test is to chip away at it, bit by bit, in order to not burn myself out learning all these new systems. With that being said, I think developing a general plan is a good starting point for today.

9/19/26

I've been looking over the material instance article and feel fairly confident I can make this property. The next thing to implement is a proper ORM texture which I plan to develop in Blender. The main thing I need to think about is exporting the texture correctly into Unreal Engine and making sure none of the data from Ambient Occlusion, Metallic or Roughness is lost.

9/19/26 Continued...

The first iteration was a success!



There are 3 main values that can be controlled, a fourth one can be added fairly easily into the emissive or specular socket. A big issue I had was trying to figure out how to actually create the material, I learned that most of the operations go through Unreal's content editor.

Blender doesn't really have a centralized file system for managing lots of assets, so this was a change I wasn't used to at all. This is also my first ever game texture I've ever created but it was a huge success!

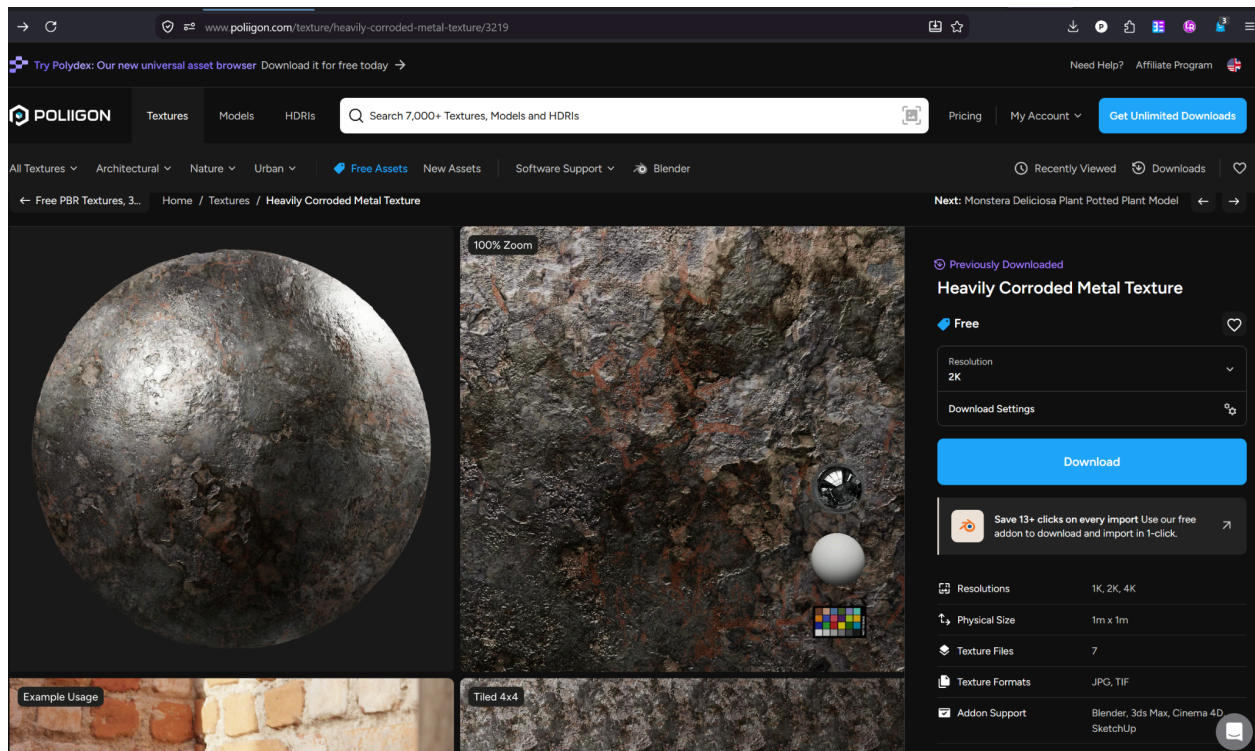
The main steps I used are as follows:

- Open the **Content Browser**
- Right click and create a '**Material**'
- Add 1 **Vector parameter**(For RGB values), and 2 **Scalar parameters**(for constant values, Roughness and Metallic are used in the above image)
- Connect the parameters to the correct sockets
- **Save the asset**, then exit back to the content browser
- Right click on the material and select '**Create Material Instance**'

I still need to import the ORM texture but the foundation is already laid out for the process.

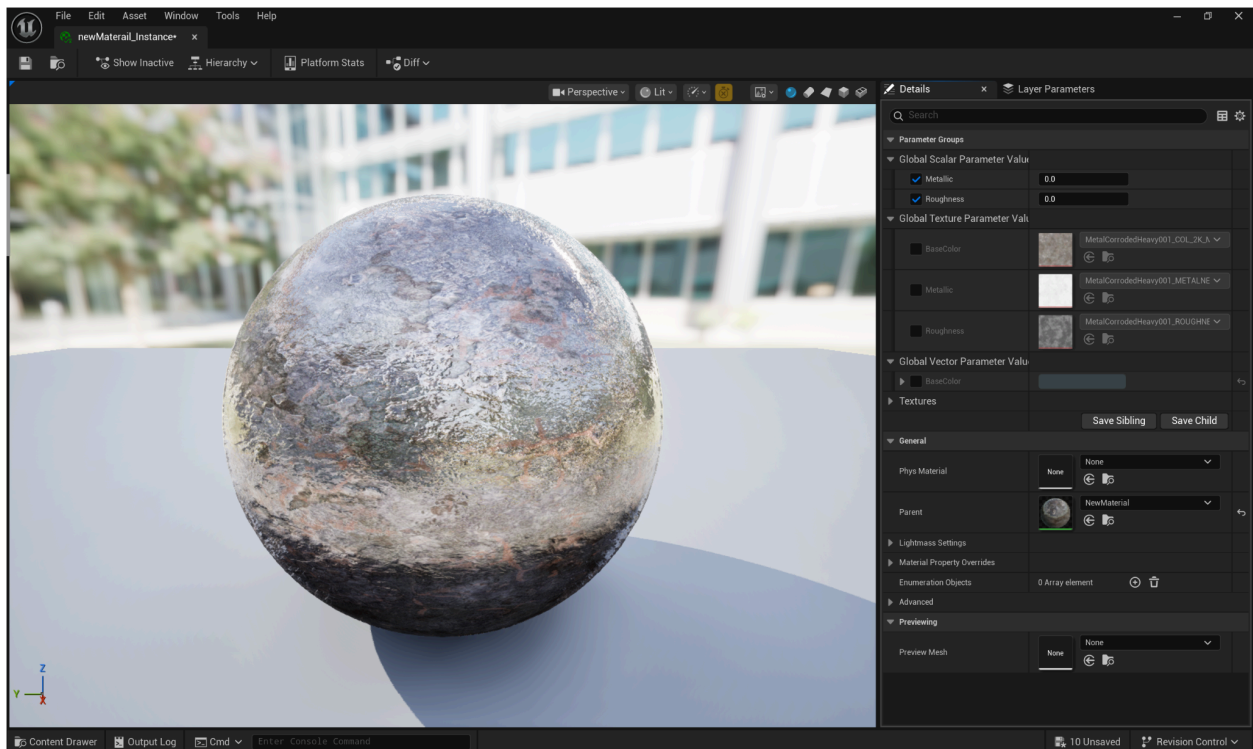
I've been running into some troubles with my texture. Unfortunately I just don't have enough experience yet to cultivate what the Skill test is asking for. Texturing materials is definitely a new realm for me and the lack of experience is leaving me a little frustrated. That being said, I've

opted to use a texture from Polligon that contains a metal texture, and from that I'll use the BaseColor and ORM Values to drive the Material instance in Unreal Engine.



Texture: <https://www.poliigon.com/texture/heavily-corroded-metal-texture/3219>

Here's the final result I got with the textures



Conclusion:

It's not perfect, but after about 2 hours of researching and learning it's still quite an achievement. I think the next steps I would need to take is to understand how Image Textures work and dig deeper into the PBR pipeline in Unreal Engine. But, it's about time to move onto something else.

Procedural Modeling

I've selected Track B → Create a procedural model with curves that takes the inputs **Spacing**, **Model Size**(Scale), and random rotation.

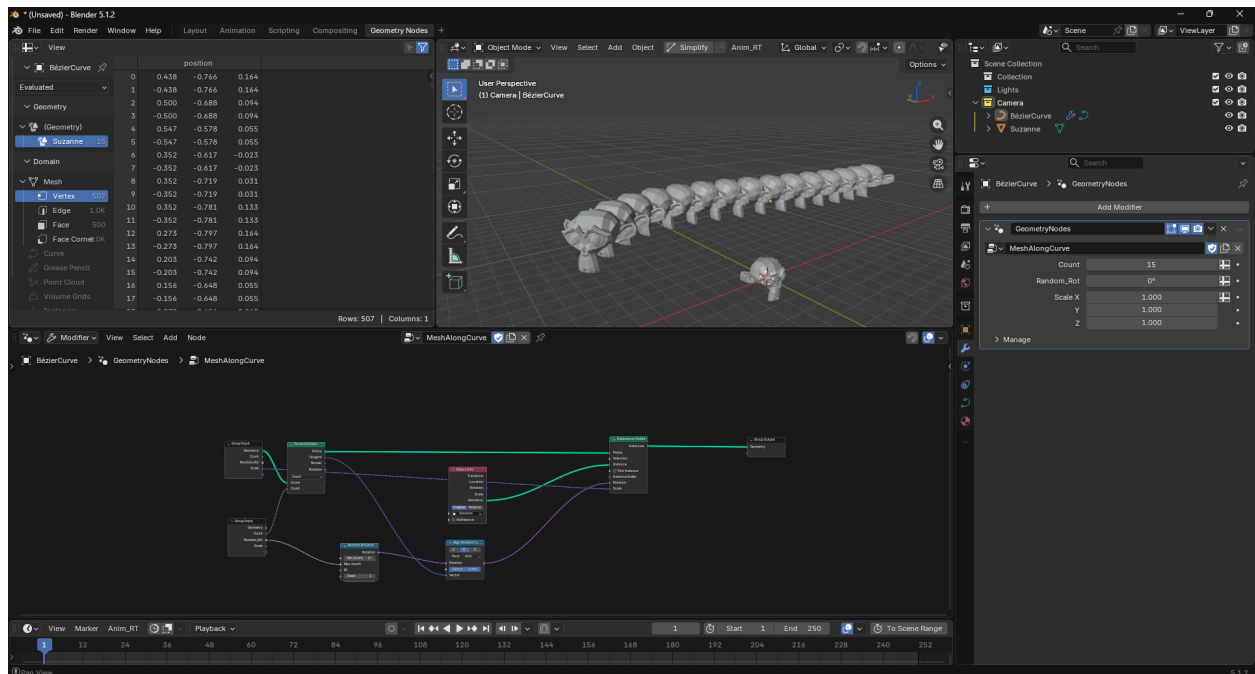
P.M Documentation

9/19/26

This is definitely going to be a Geo-Nodes project in Blender, modelling along a curve will be interesting and I still don't have that much experience with Geo-Nodes. That being said, I think most of the parameters can be achieved fairly easily through the Nodes editor. I'll see what else comes up.

9/19/26 Continued...

I've gone ahead and spent about an hour and a half on this project today, but I think it's time to call it quits. I've developed most of the criteria to a basic extent, with random rotation and scale being integrated but the 'spacing' criteria still requires some extra knowledge that I do not possess at this point.



Here's the main setup that I developed. Essentially it's just a bezier curve that has a 'Points on Curve' node which allows me to place the Suzannes(monkey heads). I found some information that suggested using the tangent of the curve would allow the Suzannes to follow the direction of the line, and I tweaked the system so that it would take Random Rotation as an input.

I also added some Group Nodes for some extra functionality for the user, so you don't have to go searching for the nodes every time. That being said, the functionality is still fairly limited and can definitely be improved in the future.



Task: Write a python script that does the following:

Scans a folder for a standard naming convention

- Ex. "S_" in front of static meshes
- For any assets that don't fit the naming convention, name them "UNASSIGNED"

Python Documentation

Documentation:

Main Skills to Learn:

- Scanning operation in Python
- Renaming in Python(Most likely a string operation)

The first article I found on searches was from w3schools:

https://www.w3schools.com/python/python_dsa_linearesearch.asp

There's also an additional term, **index**, which gets thrown around a lot with this search function idea.

Index = refers to the position of the data in the list

The main two requirements for a search function is user_input(the query) and a data_list(folder structure)

The two outcomes we want to occur is a passing of the index(if it meets the naming convention criteria) or a renaming of the index data(if it doesn't). This relationship can be expressed like this in pseudocode:

```
IF index == query:
    pass
ELSE
    Index.i = S_
    i += 1
```

Or something along those lines.

Here's an example code that I found from the article which I think highlights the idea extremely well:

```
def linearSearch(arr, targetVal):
    for i in range(len(arr)):
        if arr[i] == targetVal:
            return i
    return -1
```

```
mylist = [3, 7, 2, 9, 5, 1, 8, 4, 6]
x = 4
```

```
result = linearSearch(mylist, x)
```

```
if result != -1:
    print("Found at index", result)
else:
    print("Not found")
```

The next question on my list is how to rename items in a list, the naming convention needs to follow a certain format, but how can I rename the elements that are categorized UNASSIGNED?

Here's another article from w3schools:

https://www.w3schools.com/python/python_lists_change.asp

It actually seems to be pretty straightforward

```
myList = [1,2,3,4]
```

```
FOR items in myList:
    If 3 in myList:
        return i
    ELSE
        myList[i] = 'UNASSIGNED'
```

```
print(myList)
```

However, one of the main issues I'm running into is that you can't use myList[i] to change the values of a list. This means that my initial approach of iterating over this command will not work.

Doing a little more research, I think I've made a discovery. If you declare the value beforehand and create a loop, you can then reference the iteration value to change the values.

So for example:

```
thislist = ["apple", "banana", "cherry"]  
i = 0  
while i < len(thislist):  
    print(thislist[i])  
    i = i + 1
```

So far, this is what I've managed to develop:

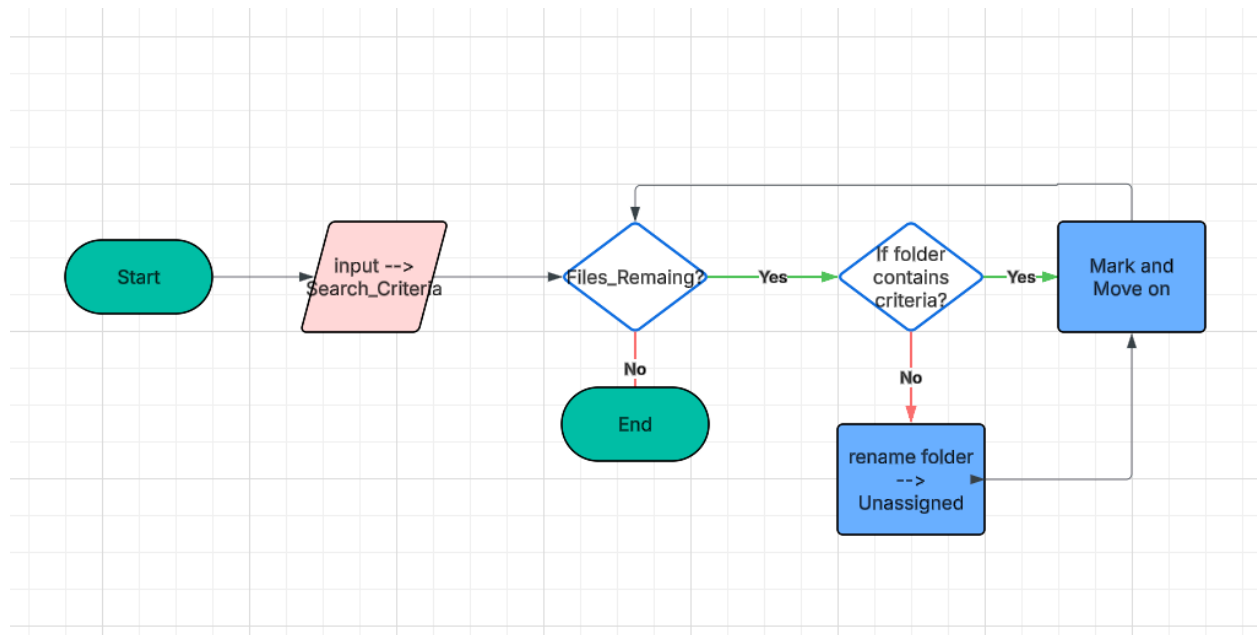
```
i = 0  
def search_function(query, data_list):  
    for index in data_list:  
        if query in data_list:  
            i += 1  
        else:  
            data_list[i] = 'UNASSIGNED'  
    return -1  
  
result = search_function('S_', ['S_',2,3,4])  
print(result)
```

The issue at the moment is that I can't call 'i' as a variable to indicate the current index, I've spent about an hour and a half breaking down the procedure but I'm running into a lot of specific errors. I think this is a good stopping point for now.

Day-by-Day Documentation:

9/17/26

For this program, I'm going to have to do a bit of research. For one, I need to figure out if I can develop a searching operation in Python, and additionally I need to be able to rename folders if they don't meet the criteria.



This is the flow diagram I've developed at the moment. I imagine this will change a lot more as I learn more about the process, but until then it's a good starting point.